EFES Project Proposal

FPGA-Assisted ECG Monitor with real-time QRS detection

Asma Rastgou
Politecnico di Torino

March 5, 2026

This report describes the design direction of an FPGA-assisted ECG monitor that combines analog sensing, microcontroller-based acquisition, and real-time digital processing on an FPGA. The project is intended as a course project for "Electronics for Embedded Systems," but it is also meant to evolve into a practical design document that records what has already been implemented, what has been verified on hardware, and what still remains to be refined.

Contents

1 Initial Proposal **4**

1.1 System Description 4

1.2 Expected Results 5

2 Component Description **6**

2.1 Analog Devices 6

2.1.1 Power Supply Regulation and Bypassing 7

2.2 Digital Components 7

2.2.1 FPGA 7

2.2.2 ADC 8

2.2.3 Microcontroller (XMC_2GO) 9

2.2.4 Clock Configuration 10

2.2.5 Flash Memory 10

2.3 Interconnections 10

2.3.1 FPGA Pin Planning 10

2.3.2 Communication Strategy 11

2.3.3 Breadboard Mapping Guide 11

2.4 Boot, Clocking, and Protocol Flow 13

2.4.1 Boot Procedure Across Devices 13

2.4.2 Clock Division and Timing Intent 13

2.4.3 Required Selection Signals and Why They Matter 14

2.4.4 How Each Protocol Works in the Current Architecture 14

2.4.5 Integrated Startup and Data-Flow Sequence 15

2.5 Verification and Testing 16

2.5.1 RTL Simulation 16

2.5.2 Breadboard Bring-Up 16

2.5.3 Practical Success Criteria 17

2.6 Current Status 18

2.6.1 Future Integration Plans 18

1 Initial Proposal

1.1 System Description

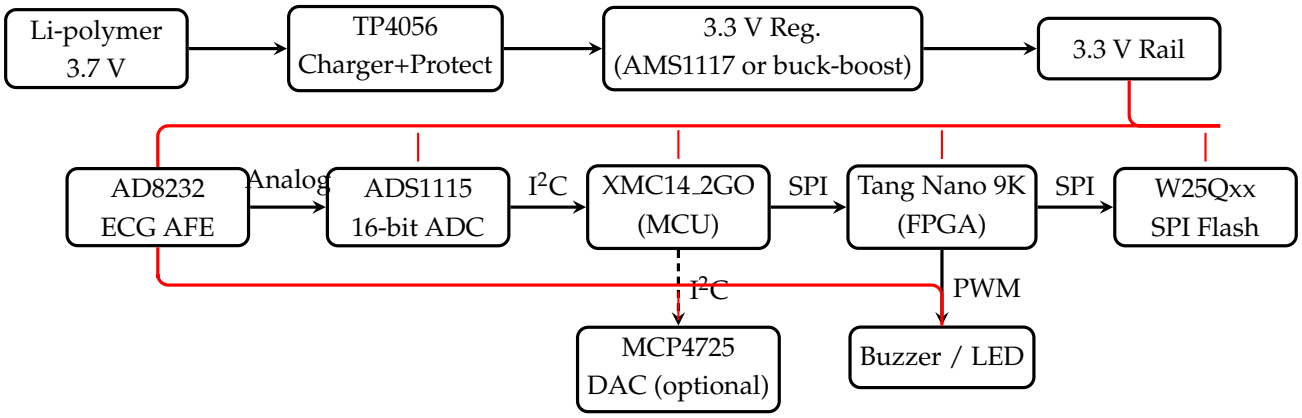
The proposed ECG monitoring system acquires and processes the cardiac signal in a complete embedded flow, combining analog sensing, mixed-signal conversion, microcontroller management, and FPGA-based processing. The signal is first captured by the **AD8232** analog front-end, which amplifies and filters the weak bio-potential obtained from the body through three surface electrodes (RA, LA, RL). The conditioned analog waveform, centered around 1.5 V and spanning a few hundred millivolts peak-to-peak, is then sampled by a **16-bit ADS1115** analog-to-digital converter.

The digitized ECG samples are read by the **XMC_1100** microcontroller over the I^2C bus. The MCU then forwards framed 16-bit samples to the **Tang Nano 9K FPGA** over SPI, where the FPGA acts as a compact streaming co-processor rather than a passive interface. In the current hardware implementation, the FPGA first receives the serial data through a clock-domain-safe SPI slave, then processes each valid sample with a sample-gated 31-tap FIR stage tuned to emphasize the QRS band. That FIR is no longer implemented as one large combinational sum; instead, it is evaluated as a short sequential multiply-accumulate process over several internal clock cycles, which makes the design much easier to close in timing while remaining far faster than the intended ECG sample rate. A lightweight threshold-based detector then estimates the occurrence of QRS complexes, and the BPM estimate is produced by a small sequential division step rather than by a single-cycle arithmetic block. This is intentionally a first working digital path: it already demonstrates real-time filtering, event detection, buffering, and actuation, while leaving room for a later extension toward a richer chain with separate high-pass, low-pass, and mains-rejection stages.

Once the samples have been filtered, the FPGA stores them temporarily in a 256-byte page buffer and writes complete pages to an external **Winbond W25Q64FV** flash memory over a second SPI interface, this time with the FPGA acting as SPI master. Abnormal BPM values drive an alarm output that can be connected either to a buzzer, to an LED, or to an MCU input for additional supervision. The board uses the on-board 27 MHz oscillator as its system clock, while the **S1** pushbutton is used as an active-low reset input, with reset release synchronized internally before the rest of the design starts operating.

Power is supplied either from USB during development or from a **3.7 V Li-ion battery** managed by a **TP4056 charger/protection circuit** and a **3.3 V linear regulator (AMS1117)** during standalone operation. The overall design forms a compact, battery-powered embedded system

that demonstrates sensing, data acquisition, digital processing, communication, storage, and actuation.



1.2 Expected Results

The final system is expected to acquire and process ECG signals in real time, detect QRS complexes in hardware, and expose the result in a form that can be observed both numerically and physically. A successful demonstration should show that the filtered signal can move through the digital chain without being lost, that the FPGA can flag heartbeat events as they occur, and that abnormal heart-rate estimates can be translated into a visible or audible alarm. Just as importantly, the project should result in a documented prototype that can be extended rather than a one-off proof of concept.

At the present stage, the most realistic short-term goals are a working breadboard prototype powered from the intended supply chain, a verified first-pass FPGA filtering and detection path, a register-level firmware prototype on the XMC14_2GO board, and repeatable logging of filtered samples to external SPI flash.

2 Component Description

2.1 Analog Devices

AD8232 Module

The AD8232 is used as the analog front-end because it is well suited to the small, noisy signals involved in ECG measurement. Its role in this project is to take the differential signal from the body electrodes and condition it into a cleaner, centered waveform that can be sampled by the ADC without needing a large amount of discrete analog circuitry. In practice, the most important characteristics are the output voltage range, the common-mode level of the signal, the amount of analog filtering already provided on the module, and the effective gain seen at the output.

Parameter	Symbol	Test Conditions/ Comments	A Grade Min	A Grade Typ	A Grade Max	W Grade Min	W Grade Typ	W Grade Max	Unit
OPERATIONAL AMPLIFIER (A1)									
Offset Voltage	V_{OS}			1	5				mV
		T_{OPR}					1	5	mV
Average TC		T_A = 0°C to 70°C		5					µV/°C
		T_{OPR}					5		µV/°C
Input Bias Current	I_B			100			100		pA
		T_A = 0°C to 70°C		1					nA
		T_{OPR}					2.5		nA
Input Offset Current	I_{OS}			100			100		pA
		T_A = 0°C to 70°C		1					nA
		T_{OPR}					1		nA
Input Voltage Range			0.1		$+V_S - 0.1$	0.1		$+V_S - 0.1$	V
Common-Mode Rejection Ratio	CMRR	V_{CM} = 0.5 V to 2.5 V		100			100		dB
Power Supply Rejection Ratio	PSRR			100			100		dB
Large Signal Voltage Gain	A_{VO}			110			110		dB
Output Voltage Range		R_L = 50 kΩ	0.1		$+V_S - 0.1$				V
		T_{OPR}				0.1		$+V_S - 0.1$	V
Short-Circuit Current Limit	I_{OUT}			12			12		mA
Gain Bandwidth Product	GBP			100			100		kHz
Slew Rate	SR			0.02			0.02		V/µs
Voltage Noise Density (RTI)	e_n	f = 1 kHz		60			60		nV/√Hz

Figure 2.1: AD8232 functional overview used as the analog front-end of the system.

Parameter	Symbol	Test Conditions/ Comments	A Grade Min	A Grade Typ	A Grade Max	W Grade Min	W Grade Typ	W Grade Max	Unit
LOGIC INTERFACE									
Input Characteristics									
Input Voltage (AC/$\overline{DC}$ and FR)									
Low	V_{IL}			1.24			1.24		V
High	V_{IH}			1.35			1.35		V
Input Voltage ($\overline{SDN}$)									
Low	V_{IL}			2.1			2.1		V
High	V_{IH}			0.5			0.5		V
Output Characteristics		LOD+ and LOD− terminals							
Output Voltage									
Low	V_{OL}			0.05			0.05		V
High	V_{OH}			2.95			2.95		V

Figure 2.2: Practical view of the AD8232 module and its connections.

2.1.1 Power Supply Regulation and Bypassing

The AD8232 is designed for low-power operation and can be supplied from a battery-based system, which makes it compatible with the intended portable use of the prototype. Even so, the analog front-end remains sensitive to supply noise, so the quality of the local power rail still matters. In a practical implementation this means keeping the 3.3 V rail stable, placing a small bypass capacitor close to the device supply pin, and avoiding unnecessary voltage excursions during charging or power-up. These details are easy to overlook during early breadboard work, but they directly affect the quality of the ECG waveform that reaches the ADC.

2.2 Digital Components

2.2.1 FPGA

The FPGA is the point in the system where raw samples become structured digital information. Its task is not simply to relay data onward, but to react continuously to the incoming stream and extract timing information from it. In the current implementation, the Tang Nano 9K receives one 16-bit sample at a time from the MCU, preserves the sample timing through a clock-domain crossing, filters the stream in fixed point, detects strong excursions associated with the QRS complex, estimates the corresponding BPM, and asserts an alarm output when the detected rate falls outside the chosen operating range. The important detail is that this is now done with timing-aware hardware blocks rather than with large single-cycle arithmetic expressions, so the current HDL is not only functionally consistent but also implementable on the target device.

The design is intentionally modular. A dedicated SPI slave receives data from the MCU. A DSP stage applies the FIR filter and the first heartbeat detector. A separate actuator block converts the BPM estimate into a simple digital alarm signal. Finally, a page buffer and flash logger cooperate to write filtered samples to the external Winbond flash in complete 256-byte bursts. This modular structure makes the HDL easier to reason about and, more importantly, makes it easier to refine one part of the system without destabilizing the rest.

Current HDL Organization

One of the most useful practical improvements in the current FPGA revision is that reset is treated as part of the architecture instead of as a loose external convenience signal. The design uses the on-board **S1** button as an active-low reset source, but its release is synchronized before the internal logic is allowed to run. In the 27 MHz core domain this produces a clean start-up for the DSP, logger, and control logic, while the SPI receiver also performs its own local reset synchronization inside the external SPI clock domain. That change avoids ambiguous start-up behavior and makes repeated testing on the board more predictable. The same attention to timing also appears in the data path: the incoming SPI words are transferred safely into the

system clock domain before the DSP pipeline consumes them, rather than being treated as if the external SPI clock were part of the same synchronous network.

FIR

A finite impulse response (FIR) filter is a natural first choice for this project because it is stable, deterministic, and straightforward to express in hardware. In discrete time, its output is a weighted sum of recent samples,

$$y[n] = \sum_{i=0}^{N} b_i \, x[n-i]$$

where each coefficient b_i weights a delayed copy of the input history. The current prototype uses a 31-tap fixed-point FIR whose coefficients were generated offline in Python and then converted to signed integers suitable for synthesis.

The implementation detail that mattered most in practice was not the equation itself, but when and how the filter state is allowed to advance. The FPGA runs from a 27 MHz clock, whereas the ECG sample rate is much lower. If the delay line were shifted on every FPGA clock edge, meaningful ECG samples would immediately be diluted by idle cycles and the filter would stop representing the real signal history. The present design therefore treats the FIR as a sample-gated block: the internal tap history advances only when a new sample arrives from the MCU. In addition, the filter is implemented as a sequential multiply-accumulate engine rather than as a single large combinational summation. That choice slightly increases the internal latency, but it dramatically shortens the critical path and still leaves a wide performance margin because the ECG sample stream is slow compared with the 27 MHz system clock. In practice, this is what transformed the FIR from a conceptually correct block into one that also meets timing on the real FPGA.

2.2.2 ADC

The ADS1115 serves as the bridge between the analog ECG front-end and the digital processing chain. It converts the conditioned output of the AD8232 into 16-bit samples that can be read reliably by the microcontroller over I^2C. In this project the practical workflow is simple: the MCU configures the converter once, points the internal register pointer to the conversion register, and then reads the two-byte conversion result at the desired sampling rate. In other words, the ADC is not conceptually complex in the final system, but it is the component that defines the raw data quality seen by every later digital block.

Limits

The most important electrical limit for the ADC in this design is that the analog input must remain inside the converter supply range. Since the AD8232 output is centered near mid-rail, roughly around 1.5 V, the PGA setting and the effective gain of the analog chain must be chosen so that peaks in the ECG waveform never drive the ADC into clipping. If the analog signal clips here, no amount of later digital processing can recover the lost information, so this is a constraint that must be respected before any FPGA algorithm becomes relevant.

2.2.3 Microcontroller (XMC_2GO)

The XMC14_2GO board acts as the acquisition and supervisory controller of the system. It reads the ADC over I^2C, packages the resulting samples, and sends them onward to the FPGA through SPI. This makes the MCU the part of the design that defines the system's sampling rhythm, while the FPGA consumes that rhythm and performs the repetitive signal processing.

From a low-level programming perspective, the board remains useful because it exposes a simple register-oriented environment for experimentation. On the XMC 2Go board, the two user LEDs are connected to port pins P1.0 and P1.1, making Port 1 a convenient place to verify basic GPIO behavior while the communication firmware is being developed. The Port 1 register block begins at address 4004_0100_H, so it offers a natural reference point for direct register-level testing before more abstract drivers are layered on top.

Register-Level Peripheral Decisions

The firmware path has been kept intentionally close to the reference manual rather than to vendor middleware: USIC channel 0 is configured for ADS1115 communication in IIC mode, USIC channel 1 is configured for the SPI links, and the periodic scheduler is generated by SysTick. In practical terms, this means that each channel is first forced to `CCR.MODE = 0000B`, then its input stages (`DXnCR`), framing (`SCTR`), and timing (`FDR/BRG/PCR`) are programmed, and only afterwards the final protocol mode is enabled. This follows the sequence recommended by the USIC chapters and avoids configuration-time glitches.

For IIC operation toward the ADS1115, the implementation uses an 8-bit transfer format with `SCTR.WLE = 7`, unlimited frame flow with `SCTR.FLE = 3F_H`, and the IIC transfer mode setting with `SCTR.TRM = 11B`. SDA and SCL are placed on P0.15 and P0.14 in open-drain alternate mode, consistent with the wired-AND bus requirement. For SPI operation, the present mapping uses P0.7 as MOSI (`USIC0_CH1.DOUT0`), P0.8 as SCLK (`USIC0_CH1.SCLKOUT`), and P0.6 as MISO input (`USIC0_CH1.DX0C`); chip-select toward FPGA and flash is currently driven manually on P0.0 and P0.9. Choosing P0.6 is not a workaround but a direct consequence of the device input matrix: it is a legal CH1 data-input source and it keeps the wiring compact on the prototype.

The MCU-to-FPGA SPI mode is not left ambiguous. The FPGA receiver HDL explicitly samples

MOSI on the rising edge and assumes idle-low SCLK, which corresponds to SPI Mode 0 (`CPOL = 0, CPHA = 0`). Therefore, the MCU side must be configured to match Mode 0 for deterministic transfers. On the SCU side, USIC clock gating is enabled through `SCU_CGATCLR0` using the same password-protection sequence used by the startup code (`SCU_GENERAL.PASSWD` unlock/write/relock). This makes clock enable robust regardless of protection state at runtime.

2.2.4 Clock Configuration

Clock configuration on the MCU matters because it affects how reliably the ADC can be polled and how precisely the outgoing SPI transactions can be timed. The XMC1100 start-up code allows the effective clock tree to be shaped through flash-programmed values that are loaded during start-up, so there is flexibility in balancing speed, peripheral availability, and power consumption. For this project, that flexibility is useful mainly because it allows the acquisition side to be stabilized first and then matched to the timing assumptions used by the FPGA design.

2.2.5 Flash Memory

As mentioned in the **Initial Proposal** section, the type of the memory used for this project is a flash memory (W25Q64FV),

2.3 Interconnections

2.3.1 FPGA Pin Planning

The physical pin assignment became an important design step once the project moved from abstract HDL to a real Tang Nano 9K board. Because an external flash memory is used, the on-board SPI flash remains untouched and the FPGA can dedicate a separate four-wire SPI bus to the Winbond device. The design also avoids the group of LCD-related pins that were intentionally left unsoldered on the module in order to keep the SD-card area accessible for later experiments.

In practice, this led to a compact and repeatable wiring plan. The 27 MHz board oscillator is connected internally to FPGA pin 52, the on-board **S1** button on pin 3 provides the active-low reset, the MCU-facing SPI interface uses pins 19, 20, and 25, the external flash uses pins 26 through 29, and the alarm output is exported on pin 30. This mapping is simple enough to wire by hand on a prototype, but structured enough that it can now be treated as part of the documented design rather than as an informal breadboard detail.

This pin plan also introduces an important electrical distinction. The reset button belongs to a 1.8 V I/O bank, so it must be constrained differently from the 3.3 V SPI and alarm signals. For that reason the reset input is described as `LVCMOS18`, while the communication and actuator pins are described as `LVCMOS33`. The Gowin `.cst` file therefore captures physical pin locations

and I/O standards, while the separate `.sdc` file captures timing constraints such as the 27 MHz system clock. In the present revision, the timing file also declares the incoming MCU SPI clock explicitly and treats the MCU SPI domain and the 27 MHz core domain as asynchronous with respect to one another. Keeping these concerns separated makes the vendor flow easier to reason about and reduces the chance of accidental constraint errors.

2.3.2 Communication Strategy

SPI is used twice in the system, but in two distinct roles. On the first SPI link, the MCU is the master and the FPGA is the slave. This path exists purely to move ECG samples into the FPGA as reliably as possible, and the HDL explicitly handles the clock-domain crossing between the external SPI clock and the internal 27 MHz clock. In the current constraint set, that external SPI clock is described separately from the core clock so that the timing tool does not misinterpret the synchronizer crossings as ordinary synchronous logic paths. On the second SPI link, the FPGA becomes the master and drives the external flash directly. In that role it issues write-enable commands, transmits page-program operations, and polls the flash status register before advancing to the next memory page.

This arrangement gives the project a useful architectural rhythm. The MCU can remain focused on acquisition and protocol handling, while the FPGA processes the stream as it arrives and commits filtered samples to non-volatile memory in complete page-sized bursts. Even in its present simplified form, this already makes the prototype feel closer to a small embedded instrument than to a collection of disconnected development boards.

2.3.3 Breadboard Mapping Guide

Because the boards are used as separate soldered modules rather than as raw ICs, the wiring should be treated as an interface-mapping exercise. The safest approach is to establish the power backbone first, then add one protocol bus at a time, and only after that connect the actuator and flash memory.

Power and Ground Backbone

All modules must share the same reference ground. In practice, this means MCU board ground, FPGA board ground, ADS1115 ground, AD8232 ground, and flash ground must be tied together before any signal wire is connected. A stable 3.3 V rail should power the ADS1115, AD8232, and flash. Place a local `100 nF` decoupling capacitor close to ADS1115 VDD–GND and another close to flash VCC–GND, plus at least one 10 μF bulk capacitor on the breadboard 3.3 V rail.

The flash control pins `/WP` and `/HOLD` should be held high (preferably through `10 kΩ` pull-ups to 3.3 V) for normal operation. The ADS1115 address pin should be tied to ground to keep its I^2C address at `0x48`, matching the firmware.

Signal Mapping

From	To	Notes
AD8232 `OUT`	ADS1115 `AIN0`	Analog ECG path. Keep this wire short and away from SPI clocks.
XMC `P0.15`	ADS1115 `SDA`	I^2C data. Add `4.7 kΩ` pull-up to 3.3 V if not already present on module.
XMC `P0.14`	ADS1115 `SCL`	I^2C clock. Add `4.7 kΩ` pull-up to 3.3 V if needed.
XMC `P0.8`	FPGA pin 19 (`SPI_SCLK_IN`)	MCU→FPGA SPI clock (Mode 0).
XMC `P0.7`	FPGA pin 20 (`SPI_MOSI_IN`)	MCU→FPGA SPI data.
XMC `P0.0`	FPGA pin 25 (`SPI_CS_N_IN`)	Active-low SPI chip select to FPGA.
FPGA pin 26	W25Q64 `CLK`	FPGA→flash SPI clock.
FPGA pin 27	W25Q64 `DI`	FPGA→flash MOSI.
FPGA pin 28	W25Q64 `DO`	Flash→FPGA MISO.
FPGA pin 29	W25Q64 `/CS`	Active-low flash chip select.
FPGA pin 30 (`ALARM_OUT`)	LED + resistor to GND	Start with LED test (`330 Ω`). Use transistor driver before connecting buzzer.

Actuator Connection Recommendation

For first hardware tests, use an LED on `ALARM_OUT` to validate threshold behavior safely. After that, if a buzzer is required, drive it through an NPN transistor (or logic-level MOSFET) with a base/gate resistor and shared ground. The FPGA pin should not be used to drive a buzzer load directly.

Bring-Up Order

The recommended physical bring-up order is: (1) power and ground checks, (2) ADS1115 over I^2C, (3) MCU→FPGA SPI link, (4) FPGA alarm LED behavior, and finally (5) flash page-write verification. This staged order reduces debugging ambiguity and makes faults easier to localize.

2.4 Boot, Clocking, and Protocol Flow

2.4.1 Boot Procedure Across Devices

At power-up, the system does not become operational in one step; each device enters a local boot phase first. The XMC1100 starts from its startup file and vector table, then executes `SystemInit()`, where core clock setup and flash wait-state preparation are performed before user firmware begins. In the present firmware structure, this is followed by `System_Init()`, which enables USIC clocking, configures IIC/SPI paths, initializes GPIO, and starts SysTick. In practice, this means peripheral registers are not safe to use until both startup and firmware-level initialization are complete.

The FPGA follows a similar two-stage idea: configuration first, operation second. The Tang Nano device loads its configuration bitstream during its own boot phase, and only after configuration and reset release does the user HDL start running. Inside the HDL, reset assertion is asynchronous but reset release is synchronized locally per clock domain (27 MHz core and external SPI clock domain). This detail matters because it prevents one domain from leaving reset in a metastable or half-initialized state while another domain is already active.

The ADS1115 and W25Q64FV do not expose a complex software boot sequence, but they still require orderly bring-up. Both must see stable 3.3 V and common ground before traffic starts, and the flash control pins `/WP` and `/HOLD` must be held inactive (logic high) for normal SPI operation. In other words, digital protocol correctness cannot compensate for incomplete power-up conditions.

2.4.2 Clock Division and Timing Intent

The project intentionally separates fast local clocks from slow physiological sampling. On the MCU side, the clock tree is configured so the processor clock and peripheral clock are deterministic for register-level timing. SysTick is then used as a scheduler source, with reload value

$$\text{RELOAD} = \frac{f_{\text{CPU}}}{250\,\text{Hz}} - 1$$

which yields a 4 ms period for a 32 MHz CPU clock. This gives a stable 250 Hz service rhythm for sampling and transfer tasks.

USIC timing is then derived from this base through `FDR` and `BRG`. For IIC, the register set is tuned for standard-mode operation with protocol pre-processor timing (`STIM`, `DCTQ`, `PDIV`) rather than ad-hoc delays in software. For SPI, the current divider choice is intentionally conservative to simplify first hardware bring-up and avoid borderline signal integrity on long breadboard wiring; once the full chain is stable, the SPI clock can be increased in a controlled way.

On the FPGA side, logic runs from the 27 MHz board oscillator, while incoming MCU SPI clock is treated as an independent asynchronous domain. For flash writes initiated by FPGA, the SPI

byte shifter generates SCLK by dividing the 27 MHz clock (through CLK_DIV), so flash bandwidth and timing margins are set by explicit HDL parameters rather than by external assumptions.

2.4.3 Required Selection Signals and Why They Matter

Several critical signals in this project are not data values but selection controls. On XMC USIC, DXnCR.DSEL selects which physical input line (A–G) feeds each stage, while INSW chooses whether the data shift unit is driven by the protocol pre-processor path or directly from the selected pin. This is why pin planning and DX selection must be consistent: a correct wire on the breadboard is still invisible to the protocol engine if DSEL points elsewhere.

For IIC, SDA/SCL are selected through CH0 input stages and open-drain output mapping, with INSW = 0 so the protocol pre-processor controls bus signaling. For SPI master on CH1, MISO is brought through DX0 input selection, while SCLK and slave-select behavior depend on BRG/PCR configuration and external/manual chip-select strategy.

At system level, slave-select signals are equally fundamental. MCU→FPGA traffic is framed by active-low CS from the MCU side. FPGA→Flash traffic is framed by FPGA-driven flash CS_N. If CS is wrong, bit-level timing can still look valid while entire words are ignored or shifted into wrong command phases.

2.4.4 How Each Protocol Works in the Current Architecture

The ADS1115 link is an IIC master-slave transaction pattern: MCU writes converter configuration (single-shot mode), polls conversion-ready status via configuration register MSB, then reads the conversion register as a two-byte sample. This keeps ADC timing explicit and avoids hidden conversion latency assumptions.

The MCU→FPGA link is SPI Mode 0 (CPOL=0, CPHA=0). The MCU sends 16-bit words MSB-first; the FPGA SPI slave samples MOSI on rising SCLK while CS is active. At the end of 16 bits, data is latched in the SPI clock domain and transferred safely into the 27 MHz domain through a request/acknowledge toggle handshake, so sample integrity is preserved across the clock-domain crossing.

The FPGA→Flash link is command-based SPI master behavior. The logger issues WREN, then page-program command plus address and payload bytes, then polls status (RDSR) until write-in-progress clears. The page buffer ensures this sequence is performed in full page bursts rather than in many small random writes, which simplifies control and better matches flash write granularity.

2.4.5 Integrated Startup and Data-Flow Sequence

The following diagram summarizes the practical sequence used by the current prototype, from first power-up to the first non-volatile page write.

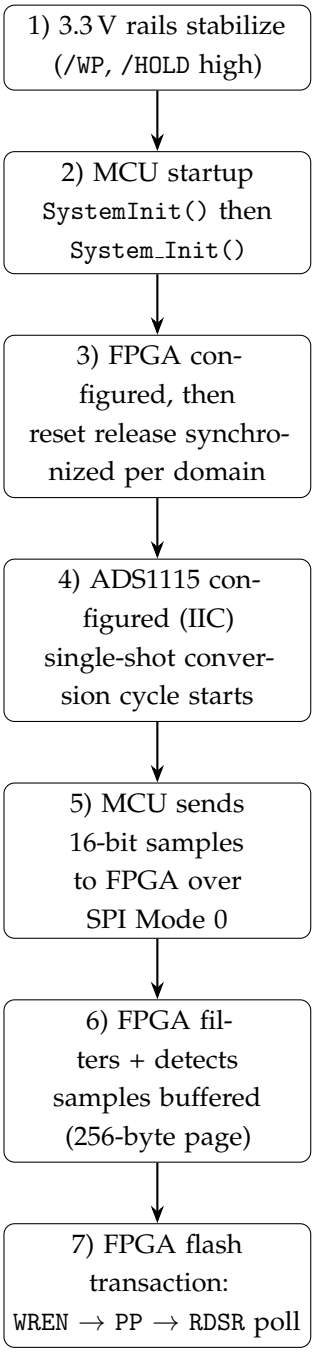


Figure 2.3: Integrated boot and protocol sequence from power-up to first flash page commit.

2.5 Verification and Testing

Because the design now exists as both HDL and physical wiring, verification has to be approached in two layers. The first layer is RTL simulation, which answers the question of whether the logic behaves coherently in a controlled and repeatable environment. The second layer is breadboard-level validation, which answers the more practical question of whether the implemented prototype behaves correctly once real clocks, wires, supply rails, and external devices are involved. Treating these as separate stages is useful: simulation is where timing relationships and protocol sequencing can be inspected in detail, while the breadboard is where electrical assumptions are exposed.

2.5.1 RTL Simulation

At the HDL level, the current repository is best understood as a set of block-level checks rather than as a complete end-to-end digital twin of the finished instrument. The existing testbenches exercise the FIR filter, the SPI byte shifter, the SPI slave receiver, and the flash page logger independently. This is enough to confirm the most important local behaviors: the FIR should only advance when a new sample is explicitly accepted, the SPI receiver should reconstruct one 16-bit word per transaction and present it cleanly in the 27 MHz domain, and the flash logger should emit the expected write-enable, page-program, and status-polling sequence on its SPI pins.

For practical waveform inspection, the most useful software path is a lightweight Verilog simulator such as `Icarus Verilog`, combined with a viewer such as `GTKWave`. In that workflow, each testbench is compiled, executed, and asked to dump a VCD waveform file so that internal signals can be examined visually rather than inferred from console messages alone. This is especially helpful for this project because several of the most important behaviors are temporal: `data_valid` should appear as a short pulse after a complete SPI word arrives, `out_valid` should only occur when the sequential FIR has completed a sample, and the flash transaction should show the expected chip-select framing around the command and data phases. A full top-level integration testbench would still be a useful future addition, but even the current module-level simulations already provide a strong way to catch functional regressions before moving back to hardware.

2.5.2 Breadboard Bring-Up

Once the simulated behavior looks reasonable, the next test is to verify the physical assembly a layer at a time instead of attempting the entire signal chain at once. The MCU-facing SPI link should be checked first: the MCU clock, data, and chip-select lines are wired to FPGA pins 19, 20, and 25 respectively, while the flash-facing SPI bus is driven from FPGA pins 26 through 29 toward the external W25Q64FV. The flash itself must also be given a stable 3.3 V supply, a common ground, a local bypass capacitor, and logic-high levels on the `/WP` and `/HOLD` pins.

Only after these static electrical conditions are correct does it make sense to interpret protocol activity.

In practice, the most informative hardware instrument at this stage is not an oscilloscope but a small logic analyzer connected to both SPI buses. If the MCU sends a short sequence of known 16-bit words, one can first confirm that those words are visibly clocked into the FPGA interface. The present design stores filtered samples in a 256-byte page buffer, which means that a flash write is not expected immediately; it should begin only after 128 sixteen-bit samples have been accepted. For that reason, the absence of flash activity after just a few samples is not necessarily an error. Once the page buffer fills, however, the flash chip-select line should fall, the write-enable and page-program commands should appear on the bus, and the device should be polled until the internal write cycle completes. This sequence is far easier to trust when it has been observed directly on the wires rather than only assumed from the source code.

At MCU firmware level, two quick register checks are especially useful during bring-up. First, after calling the USIC clock-enable routine, `SCU_CGATSTAT0.USIC0` should confirm that gating has actually been cleared; this prevents silent failures caused by protection-state assumptions. Second, a short SPI test frame observed on the analyzer should show idle-low SCLK and sampling on rising edges, matching Mode 0 as required by the FPGA SPI slave implementation. If these two conditions hold, the remaining debugging effort usually shifts from low-level register setup to protocol sequencing and application logic.

2.5.3 Practical Success Criteria

The project can be considered to be working at a meaningful prototype level when three conditions are simultaneously true. First, simulation should show that the individual digital blocks behave in a stable and repeatable way, especially at the points where validity strobes and state transitions matter. Second, the breadboard should show clean and correctly framed SPI traffic on both the MCU-facing and flash-facing interfaces, with the expected delay between incoming samples and a full page write. Third, the external flash should be read back after a test run so that the programmed page can be verified at the configured starting address. That readback is particularly important because it converts a visible bus transaction into proof that non-volatile storage actually changed state.

From a system perspective, the most convincing demonstration is therefore not a single blinking LED but a short chain of evidence: known sample words enter from the MCU, the FPGA accepts them, the internal filter and detector process them, the page buffer eventually triggers a flash transaction, and the stored bytes can later be recovered from the Winbond device. If that chain can be repeated reliably, then the prototype is no longer merely syntactically correct HDL or a successful place-and-route result; it is a functioning embedded system with an observable and testable data path.

2.6 Current Status

At the current stage, the HDL design can be synthesized and routed successfully with the Gowin toolchain for the Tang Nano 9K target, and it now satisfies the timing requirements of the 27 MHz system clock. Earlier revisions were functionally plausible but too arithmetic-heavy in a single cycle, which led to large negative slack in the timing report. After restructuring the FIR as a sequential multiply-accumulate block, moving the BPM estimation away from a single-cycle divider, and making the clock-domain assumptions explicit in the timing constraints, the design now closes timing with positive slack. In practical terms, the current implementation reaches an internal maximum frequency above the 27 MHz target, so the design is no longer merely synthesizable; it is also implementable with margin on the intended device.

In parallel, the MCU firmware project now builds successfully in the DAVE ARM-GCC flow and produces an `.elf` image. The recent build report confirms a complete link step and a compact footprint (`text = 3112`, `data = 4`, `bss = 1052`, total `4168` bytes). This is an important milestone because it confirms that the current register-level codebase, startup files, and linker setup are consistent enough to generate runnable firmware. At the same time, it should be interpreted as a build-level success rather than final functional proof: protocol-level validation on hardware (I^2C transactions, SPI mode matching, and flash readback consistency) remains the next decisive step.

The result is not yet the final signal-processing architecture envisioned at the beginning of the project, but it is a coherent and testable baseline: data can enter the FPGA from the MCU, pass through a real fixed-point filter, trigger a simple heartbeat detector, raise an alarm when needed, and be written to an external SPI flash device. That working baseline is important, because it turns the rest of the project from an abstract proposal into an iterative engineering task.

The next natural step is to refine the digital signal-processing path itself. The present detector is deliberately lightweight so that the full acquisition, buffering, and storage chain could be made to work first. Once that foundation is stable on hardware, the FIR stage can be expanded into a richer filtering chain and the QRS detector can be replaced by a more faithful implementation of the desired algorithm.

2.6.1 Future Integration Plans

For the current project scope, the alarm path (threshold comparison plus LED/buzzer actuation) remains the primary demonstrator. However, a waveform-visualization extension is planned as a follow-on activity because it would make the platform much more useful for signal-level validation and personal experimentation.

The most direct extension is a real-time telemetry path from FPGA to a PC. In practical terms, this means adding a lightweight UART transmitter in the FPGA, packetizing selected data (for example raw sample, filtered sample, and BPM), and forwarding it to a serial USB bridge for live plotting in Python or MATLAB. A second, more advanced extension is an offline path

where data already stored in flash is read back and streamed to the host on request. That version requires a flash-read command sequence in addition to the currently implemented page-program flow.

These additions are intentionally deferred, not rejected. Keeping them in the roadmap clarifies that the present design is a stable base system, while waveform observability is a planned integration phase rather than an abandoned requirement.